NATIONAL UNIVERSITY OF MODERN LANGUAGES

SOFTWARE QUALITY ENGINEERING LAB REPORT – 9

NAME: Abdul Rafay

ROLL NO: FL23791

PROGRAM: BSSE (6th Semester)

COURSE: SOFTWARE QUALITY ENGINEERING

SUBMITTED TO: Mam Rubab

DATE: 9 May 2026 **DAY: Saturday**

Task 1: Testing Levels

Testing levels define the stage at which testing is performed during the software development lifecycle. Each level focuses on a different scope of the system.

Unit Testing

Unit testing means testing a single small function or module on its own, without involving other parts of the system.

#	Example
1	Test the password validation function to check if it correctly rejects passwords shorter than 8 characters.
2	Test the email format checker to verify it returns an error for inputs like 'abc@' or 'usermail.com'.

Integration Testing

Integration testing checks whether two or more modules work correctly when connected together.

#	Example
1	Test the login module with the database — when a user enters valid credentials, the system should correctly fetch and match data from the database.
2	Test the course registration module with the confirmation module — after a student selects a course, the confirmation message should be triggered correctly.

System Testing

System testing tests the complete application from start to finish as one whole system.

#	Example
1	A new student registers an account, logs in, selects a course, and receives a confirmation message — the entire flow is tested end to end.

#	Example
2	Test that the logout feature works correctly after a full session of registration and course selection.

Acceptance Testing (UAT)

Acceptance testing is done from the end user's point of view to check whether the system meets real requirements.

#	Example
1	A real student tries to register and enroll in courses and confirms the process is straightforward without any technical assistance.
2	A university administrator verifies that the system displays correct course confirmations and student details as described in the requirements.

Task 2: Testing Types

Functional Testing

Functional testing checks whether each feature of the system works as expected according to the requirements.

Registration: Enter valid name, email, password, and program name. The system should create the account and redirect to the login page.

Login: Enter correct email and password. The system should log the user in and display the dashboard with the student name.

Invalid Login: Enter a wrong password. The system should display an error message such as Invalid credentials.

Course Registration: Select an available course and click Register. The system should save the selection and display a confirmation message.

Non-Functional Testing

Type	Example
Performance	100 students log in at the same time — the system should respond within 3 seconds without crashing.
Security	Enter SQL injection input in the login field — the system should block it and deny access.
Usability	A first-time user should be able to complete registration without any guidance — the interface should be clear and self-explanatory.
Reliability	Run the system continuously for 8 hours — it should not crash, freeze, or lose any student data.

Regression Testing

Regression testing is done after a new feature is added to make sure the existing features still work correctly.

Scenario 1: After adding the Edit Profile feature, run the login test again to confirm it still works as before.

Scenario 2: After adding profile editing, test the course registration feature again to make sure it still saves correctly and shows the confirmation message.

Smoke Test

A smoke test is a quick basic check to see if the system is stable enough for detailed testing.

Scenario: Open the application, check that the registration page loads, check that the login page loads, and verify the homepage is accessible. If these basic checks pass, the build is considered stable and ready for further testing.

Sanity Test

A sanity test is a focused check performed after a specific bug is fixed to confirm that the fix works correctly.

Scenario: After fixing the bug where the confirmation message was not appearing, log in with valid credentials, register for a course, and verify that the confirmation message now displays correctly.

Task 3: Test Cases

The following table contains 10 test cases including both positive and negative scenarios. Each test case includes test steps, test data, expected result, actual result, status, and remarks.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status	Remarks
TC-01	Valid user registration	1. Open Register page 2. Fill all fields 3. Click Register	Name: Ali Ahmed Email: ali@gmail.com Password: Ali@1234 Program: SE	Account created, redirected to login page	Account created and redirected to login	Pass	All fields accepted correctly
TC-02	Registration with missing email	1. Open Register page 2. Leave email empty 3. Click Register	Name: Ali Email: (blank) Password: Ali@1234	Error: Email is required	Error message displayed	Pass	Validation working
TC-03	Valid login	1. Open Login page 2. Enter email and password 3. Click Login	Email: ali@gmail.com Password: Ali@1234	Dashboard loads and student name is visible	Dashboard loaded with name	Pass	Login working correctly
TC-04	Login with wrong password	1. Open Login page 2. Enter correct email, wrong password	Email: ali@gmail.com Password: wrong123	Error: Invalid credentials	Error message displayed	Pass	Negative case handled

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status	Remarks
		3. Click Login					
TC-05	Login with unregistered email	1. Open Login page 2. Enter email not in system 3. Click Login	Email: unknown@test.com Password: Test@123	Error: Account not found	Error message displayed	Pass	System rejects unknown users
TC-06	Valid course registration	1. Login 2. Go to Courses 3. Select a course 4. Click Register	Logged in student, available course	Confirmation: You have been registered for this course	Confirmation message displayed	Pass	Course registration working
TC-07	Course registration without selection	1. Login 2. Go to Courses 3. Click Register without selecting	No course selected	Error: Please select a course first	Error message displayed	Pass	Validation working
TC-08	Registration with weak password	1. Open Register page 2. Enter short password 3. Click Register	Password: 123	Error: Password must be at least 8 characters	Error message displayed	Pass	Password rule enforced
TC-09	Logout functionality	1. Login with valid credentials	Active user session	User logged out,	Redirected to login page	Pass	Logout working correctly

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status	Remarks
		2. Click Logout button		redirected to login page			
TC-10	Registration with invalid email format	1. Open Register page 2. Enter invalid email 3. Click Register	Email: aliATgmail.com	Error: Please enter a valid email address	Error message displayed	Pass	Email format validation working

Task 4: Automation Planning

Test Cases Selected for Automation

The following five test cases were selected for automation because they are executed repeatedly after every code change or release. Their steps are fixed, predictable, and do not require any human judgment.

Auto TC ID	Manual TC Ref	Feature	Automation Tool	Reason for Automation
ATC-01	TC-03	Valid Login	Selenium / Cypress	This test runs after every build and has fixed, repeatable steps — ideal for automation.
ATC-02	TC-04	Login with Wrong Password	Selenium / Cypress	Error message verification is consistent and quick to automate.
ATC-03	TC-01	Valid Registration	Cypress / Playwright	Core feature that must be verified after every update.
ATC-04	TC-06	Course Registration	Selenium	Used by all students and must be confirmed working at every release.

Auto TC ID	Manual TC Ref	Feature	Automation Tool	Reason for Automation
ATC-05	TC-09	Logout	Cypress	Simple, repetitive steps — a straightforward candidate for automation.

Why These Test Cases Should Be Automated

These five test cases were selected for automation because they are performed repeatedly after every code change or new release. The steps involved are fixed and do not require any human judgment or visual inspection. They cover the most critical features of the system, meaning any failure in these areas would be a serious problem. Automating them saves significant time compared to running them manually every time.

Pseudocode for Automated Test Case — TC-03 (Valid Login)

```
Test Name: Valid Login Test

Step 1:  Open the browser
Step 2:  Navigate to the application login page URL
Step 3:  Locate the email input field and enter ali@gmail.com
Step 4:  Locate the password input field and enter Ali@1234
Step 5:  Click the Login button
Step 6:  Wait for the page to load completely
Step 7:  Check that the current URL contains /dashboard
Step 8:  Check that the student name Ali Ahmed is visible on the screen
Step 9:  If both checks pass, mark the test as PASSED
Step 10: If either check fails, mark the test as FAILED and print the
error
```

Task 5: Final Analysis

Benefits of Automated Testing for This System

Automated testing saves a large amount of time because test scripts run within seconds compared to manually going through each step. The results are always consistent since scripts follow the exact same steps every time without making mistakes. Automated testing is especially useful for regression testing because after every update, all previously written test cases can be re-run automatically without any manual effort. It also allows tests to run overnight or on weekends without anyone present. For a system like the Student Registration System, where the same core features are tested repeatedly, automation reduces workload significantly over time.

Limitations of Automated Testing for This System

Writing automation scripts requires time and technical skill upfront, which means it is not always the fastest option for quick one-time checks. Automated tests cannot evaluate how the interface looks or whether it is easy to use from a student's perspective, since scripts only check data and outcomes, not visual design or user experience. If the user interface changes, for example if a button is renamed or moved, the existing scripts will break and need to be updated. Automation is also not suitable for exploratory testing where the tester is looking for unexpected or unusual bugs that were never anticipated.

Conclusion

Both manual and automated testing are necessary for a complete testing strategy. Manual testing works best when a feature is new, when the interface needs to be evaluated visually, when tests will only be performed once, or when the tester needs to explore the system freely to find unexpected issues. Automated testing works best when the same test needs to run repeatedly after every update, when the steps are fixed and predictable, and when fast results are needed without human involvement. For the Student Registration System, the best approach is to automate all repetitive and critical test cases such as login, registration, and logout, while using manual testing for usability checks and any new features that are still being developed.